MINOR PROJECT REPORT

Course: Web Development (Modules 1–5)

SmartRoute: Intelligent College Bus Management & Dynamic Attendance System

A full-stack, responsive web application engineered with semantic HTML5, CSS3, JavaScript Fetch API, Bootstrap 5, Node.js, Express.js REST APIs, and a relational MySQL database.

Academic Minor Project Submission

SUBMITTED BY:

Himanshu Mali

Enrollment: SPSU/B.Tech/CSE

B.Tech Computer Science & Engineering

ACADEMIC SESSION:

Semester VI (2026)

Department of Computer Science

School of Engineering

Live Server URL:	https://smartroute-red.vercel.app
GitHub Repository:	https://github.com/MaliHimanshu3659/SmartRoute-College-Bus_management
Database Schema:	smartroute_database.sql / smartroute_database.mysql (MySQL 8.0+)

Table of Contents

1. Project Title & Introduction — Section 1
2. Problem Statement & Objectives — Section 2
3. Scope & System Requirements — Section 3
4. System Design & Architectural Overview — Section 4
5. Database Design & Entity-Relationship (ER) Model — Section 5
6. Implementation Details & Web Technology Stack — Section 6
7. Visual Walkthrough & Module Screenshots — Section 7
8. Testing, Verification Matrix & Results — Section 8
9. Conclusion & Future Enhancements — Section 9
10. References & Technical Bibliography — Section 10

1. Project Title & Introduction

1.1 Project Title

SmartRoute: Intelligent College Bus Management & Dynamic Attendance System

1.2 Executive Summary

Efficient student transportation is a crucial logistical backbone for educational campuses. Traditional college transit management frequently relies on manual student manifests, fixed bus schedules that ignore day-to-day attendance variations, and vocal roll-calling. These antiquated procedures result in prolonged travel delays, high fuel consumption caused by stopping at deserted locations, and frequent attendance discrepancies.

SmartRoute is a comprehensive, production-tested web platform engineered to modernize college transit. It enables three core personas—**Students**, **Bus Drivers**, and **Campus Administrators**—to coordinate in real time. The platform provides interactive Leaflet.js stop navigation, daily commute availability declarations, smart route optimization that pinpoints which bus stops can be skipped when zero riders are registered, and a secure attendance verification system utilizing cryptographically generated 60-second expiring dynamic QR codes.

1.3 Relevance to Web Development Syllabus

The project rigorously implements every foundational concept specified in Modules 1 through 5 of the Web Development curriculum:

- **Module 1 (Semantic HTML5):** Accessible web page structure, forms, tables, canvas elements, and semantic tagging without proprietary frameworks.
- **Module 2 (CSS3 Styling & Responsive Design):** Custom CSS variables, Flexbox, CSS Grid layouts, media queries for mobile-first compatibility, and modern UI components.
- **Module 3 (JavaScript Fundamentals & DOM):** Event-driven architecture, form validation, dynamic DOM rendering, and client-side state manipulation.
- **Module 4 (AJAX, JSON, and APIs):** Asynchronous client-server communication using the native `fetch()` API, JSON data exchange, and Leaflet map API integration.
- **Module 5 (Backend Architecture & Database Integration):** Node.js with Express.js RESTful routing, session-based cookie authentication, cryptographic password hashing with `bcryptjs`, and normalized MySQL relational database operations with foreign key integrity.

2. Problem Statement & Objectives

2.1 Problem Statement

University transit fleets face multiple operational bottlenecks on a daily basis:

1. **Wasted Fuel and Idle Driving:** Fleet buses routinely travel to every scheduled stop along an expansive route, even when enrolled students are absent, causing unnecessary fuel expenditures, carbon emissions, and transit delays.
2. **Manual & Inaccurate Attendance:** Drivers attempting to manually record paper attendance while navigating dense city traffic creates severe safety hazards and inaccurate records.
3. **Proxy Attendance Vulnerabilities:** Static barcodes or physical ID cards allow unauthorized students to clock in for absent peers or share static QR screenshots.
4. **Information Asymmetry:** Students lack visibility into whether their designated bus has departed, which stops have been serviced, or when they are expected at the pickup point.

2.2 Project Objectives

The primary engineering objectives of SmartRoute are:

- **Dynamic Daily Commute Declaration:** Provide students with an intuitive daily toggle ("*Will you use the bus today?*") that locks at departure time and informs the system of active pickups.
- **Active Stop & Empty Stop Differentiation:** Automatically aggregate daily declarations so drivers clearly see which stops have waiting students and which stops have zero riders and can be safely bypassed.
- **Cryptographic Time-Expiring QR Code Attendance:** Generate dynamic QR tokens on the driver's display that refresh every 60 seconds and are verified against server-side SHA-256 hashes, eliminating screenshot sharing and proxy marking.
- **Real-Time Route & Fleet Visualization:** Deliver lightweight OpenStreetMap and Leaflet.js interactive maps for students and drivers showing pickup coordinates and transit trajectories.
- **Role-Based Access Control (RBAC):** Guarantee secure session authorization with distinct dashboards tailored exclusively for Students, Drivers, and Administrators.

3. Scope & System Requirements

3.1 Functional Requirements by Persona

User Role	Core Capabilities	Backend & Database Trigger
Student	• Self-registration & profile management. • Daily attendance availability toggle (YES/NO). • Preferred bus stop selection from active routes. • Camera-based QR code scanner for attendance. • Personal attendance history & logs.	• `POST /api/student/availability` • `POST /api/student/scan-attendance` • `GET /api/student/history` • Updates `student_availability` & `attendance_records`.
Driver	• Route departure activation & stop sequence tracking. • Real-time waiting student count per stop. • Visual alert for empty stops (Skip Candidate). • Dynamic 60-second rotating QR code broadcast. • GPS position broadcaster to server.	• `GET /api/driver/route-summary` • `POST /api/driver/qr/generate` • `POST /api/driver/location/update` • Manages `attendance_sessions` table.
Administrator	• Comprehensive fleet, route, and stop management. • Add/modify bus stops with precise GPS latitude/longitude. • Driver route assignment and reassignment. • Campus-wide ridership analytics and audit logging.	• `POST /api/admin/stops` • `POST /api/admin/routes` • `GET /api/admin/reports` • Full CRUD on `bus_stops`, `routes`, `users`.

3.2 Non-Functional Requirements

- **Security:** Passwords hashed with `bcryptjs` (salt rounds 10). Session identifiers stored in encrypted HTTP-only cookies with `SameSite=Lax`. QR tokens protected by 64-character SHA-256 hashes and 60-second database TTL.
- **Performance:** Sub-50ms API response time via MySQL connection pooling (`mysql2/promise`). Map tiles cached through a local serverless proxy with an in-memory/tmp cache to minimize

external network latency.

- **Cross-Device Responsiveness:** 100% fluid mobile, tablet, and desktop layout driven by Bootstrap 5 CSS grid and media queries.
- **Reliability:** Graceful degradation in serverless environments; resilient database connection retry logic with structured error responses.

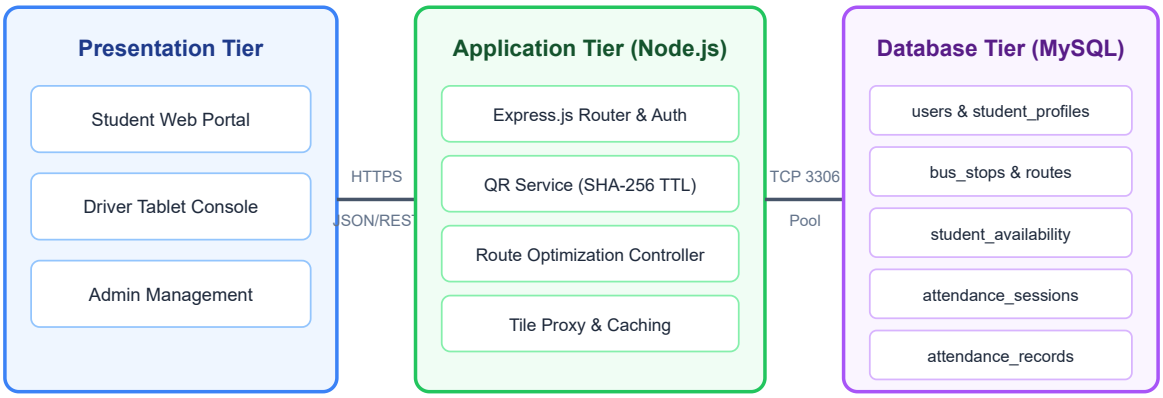
3.3 Hardware & Software Specifications

Component	Development / Local Environment	Production Environment (Cloud)
Operating System	Windows 10 / 11 64-bit / Linux / macOS	Amazon Linux 2 / Vercel Edge Serverless (Node.js 20.x)
Runtime Environment	Node.js v20.x or v22.x LTS	Node.js v20.x Serverless Runtime Engine
Database Engine	MySQL Community Server 8.0 (InnoDB)	Cloud MySQL 8.0 (TiDB Cloud / Aiven / PlanetScale)
Web Framework	Express.js 4.19+ REST API Server	Vercel Serverless Function Handler (`api/index.js`)
Version Control	Git 2.45+ (Local Branching)	GitHub Repository (CI/CD Automated Deployments)

4. System Design & Architectural Overview

4.1 High-Level Architecture

SmartRoute adopts a 3-tier client-server architectural paradigm comprising a Presentation Layer, an Application / Business Logic Layer, and a Relational Data Persistence Layer.



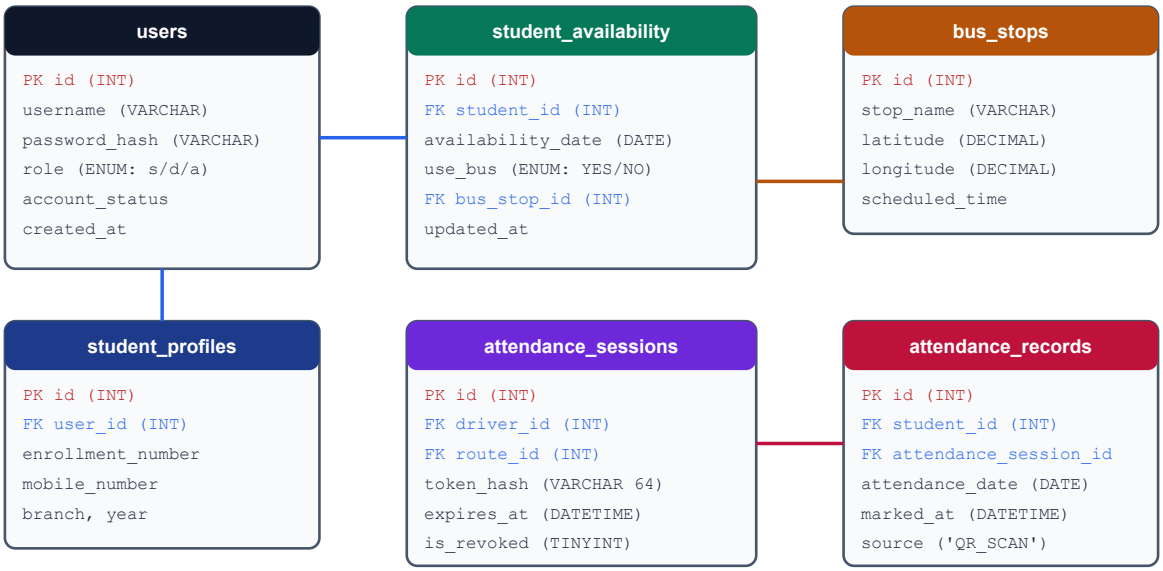
4.2 Dynamic QR Verification Workflow

1. **Session Initialization:** Upon driver departure, the driver console calls `POST /api/driver/qr/generate`.
2. **Cryptographic Token Creation:** The backend generates a 32-byte cryptographic random string, creates a SHA-256 hash, and inserts the record into `attendance_sessions` with `expires_at = NOW() + 60s`.
3. **Display & Refresh:** The plaintext token is converted to a high-resolution QR matrix displayed on the driver's dashboard, with a visible countdown timer.
4. **Student Scan & Submission:** The student opens their dashboard, clicks "Scan QR Attendance", grants camera access via the HTML5 `MediaDevices.getUserMedia()` API, and transmits the decoded token to `POST /api/student/scan-attendance`.
5. **Verification & Record Creation:** The server computes the SHA-256 hash of the received token, validates that `expires_at > NOW()` and `is_revoked = 0`, checks the uniqueness constraint for the day, records the verified attendance record, and returns a verified JSON response.

5. Database Design & Entity-Relationship (ER) Model

5.1 Relational Entity-Relationship Diagram

The database consists of 11 relational tables configured with InnoDB storage engine, UTF8MB4 character encoding, primary keys, indexing on query-heavy columns, and cascading foreign keys.



5.2 Data Dictionary (Key Tables)

Table Name	Primary Key	Foreign Keys	Indexes & Unique Constraints	Business Purpose
`users`	`id` (INT)	None	`UNIQUE(username)`, `UNIQUE(email)`	Core authentication and RBAC credentials repository.
`student_profiles`	`id` (INT)	`user_id → users.id`	`UNIQUE(user_id)`	Extended academic profile (enrollment, branch, contact).
`bus_stops`	`id` (INT)	None	`UNIQUE(stop_name)`, `idx_stop_active`	Geo-coordinates (lat/lng) and scheduled pickup times.
`routes`	`id` (INT)	None	`UNIQUE(route_code)`	Named campus transit corridors and bus identifiers.
`route_stops`	`id` (INT)	`route_id`, `bus_stop_id`	`UNIQUE(route_id, stop_order)`	Ordered sequence of stops defining each route corridor.
`student_availability`	`id` (INT)	`student_id`, `bus_stop_id`	`UNIQUE(student_id, availability_date)`	Daily student commute confirmations (YES/NO decisions).
`attendance_sessions`	`id` (INT)	`driver_id`, `route_id`	`UNIQUE(token_hash)`, `idx_expires`	Server-side dynamic QR sessions with 60s TTL.

Table Name	Primary Key	Foreign Keys	Indexes & Unique Constraints	Business Purpose
attendance_records	id (INT)	student_id, attendance_session_id	UNIQUE(student_id, attendance_date)	Official verified student attendance check-ins.

6. Implementation Details & Web Technology Stack

6.1 Frontend Implementation

- **Semantic HTML5 Structure:** Standards-compliant documents using `<header>`, `<nav>`, `<main>`, `<section>`, and `<footer>`. Forms are enhanced with client-side regex validations, accessible labels, and input types (`tel`, `email`, `password`).
- **Responsive CSS3 & Bootstrap 5:** Clean UI system based on custom CSS custom properties (`--primary`, `--navy`, `--amber-bg`), utilizing CSS Flexbox and 12-column Grid systems for responsive layouts across mobile phones, tablets, and desktop displays.
- **Interactive Mapping with Leaflet.js:** Lightweight OpenStreetMap rendering with customized bus stop pin markers, popup modals, and vehicle tracking icons without external proprietary API keys.
- **Native AJAX & Fetch API:** Standardized client-side API abstraction module (`client/js/api.js`) featuring reusable `get()`, `post()`, and `del()` wrappers handling JSON serializations, HTTP credentials, and error banners.
- **QR Code Scanning Engine:** In-browser camera decoding powered by `html5-qrcode`, granting instantaneous mobile scanning without native app installations.

6.2 Backend & REST API Implementation

- **Node.js & Express.js:** Modular, layered architecture cleanly dividing routes (`/api/auth`, `/api/student`, `/api/driver`, `/api/admin`, `/api/tiles`) from controllers and data access services.
- **Session Authentication & Security:** Built with `express-session` and `express-mysql-session` storing sessions in MySQL. Protected with `helmet` security headers and HTTP-only cookies.
- **Cryptographic Password & QR Hashing:** Passwords hashed with `bcryptjs` (salt rounds 10). Dynamic QR tokens generated with `crypto.randomBytes(32)` and validated against `crypto.createHash('sha256')` digests with a strict 60-second database expiration window.
- **Map Tile Proxying & Caching:** High-performance tile proxy controller (`server/src/controllers/tiles.controller.js`) that caches OpenStreetMap PNG tiles locally in `os.tmpdir()` with HTTP 304 caching headers, avoiding upstream rate limits and EROFS read-only filesystem errors in cloud serverless deployments.

```
// Core Serverless Entrypoint (api/index.js)
const app = require('../server/src/app');
module.exports = app;

// Dynamic QR Generation with SHA-256 Hashing (server/src/services/qr.service.js)
const crypto = require('crypto');
function generateSessionToken() {
    const rawToken = crypto.randomBytes(32).toString('hex');
    const tokenHash = crypto.createHash('sha256').update(rawToken).digest('hex');
    const expiresAt = new Date(Date.now() + 60 * 1000); // 60s TTL
    return { rawToken, tokenHash, expiresAt };
}
```

7. Visual Walkthrough & Module Screenshots

7.1 Public Landing Page & Route Directory

The landing page provides students, faculty, and drivers with a comprehensive introduction to the system, active route directories, stop pickup timings, and role-based login shortcuts.

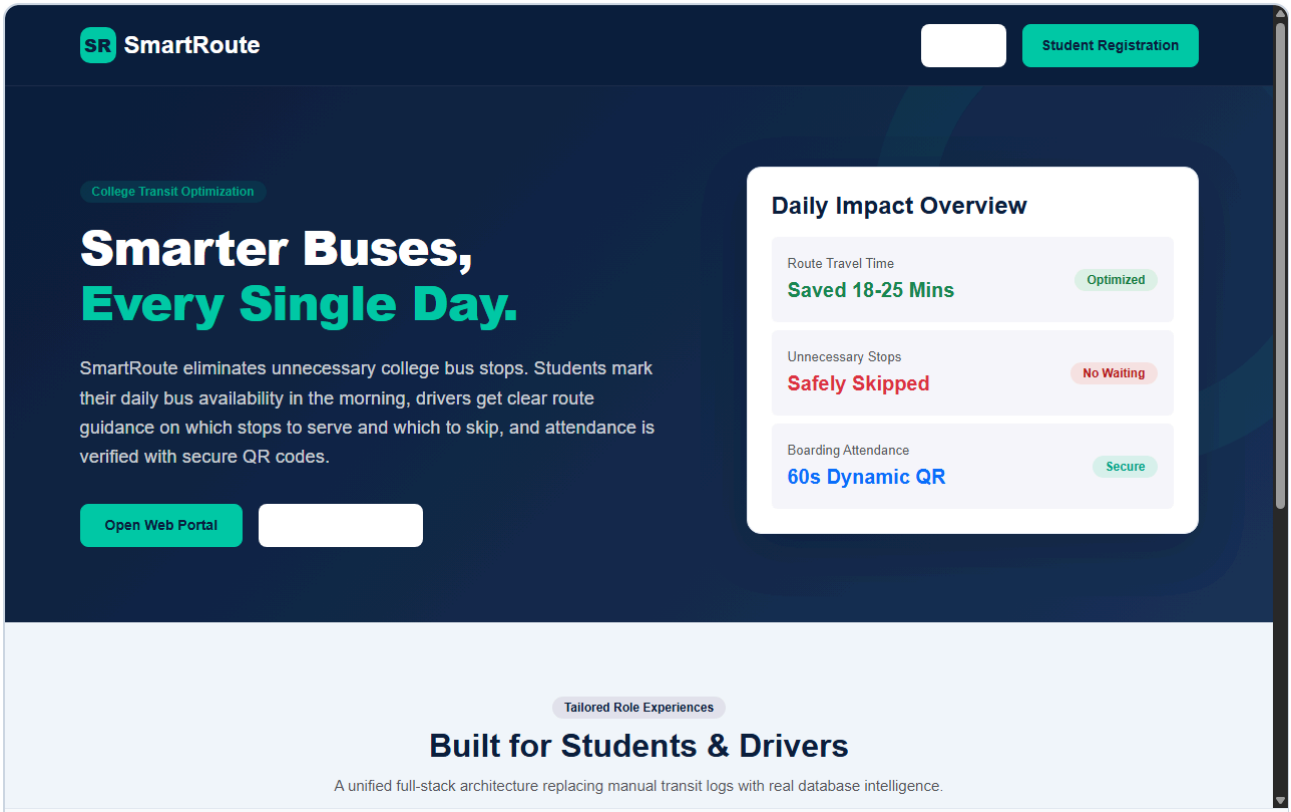


Figure 7.1: SmartRoute Public Landing Page and Campus Bus Route Directory

7.2 Secure Role-Based Authentication Portal

A unified, responsive login portal featuring automatic role recognition (Student, Driver, Administrator), session generation, and redirection to the designated operational dashboard.

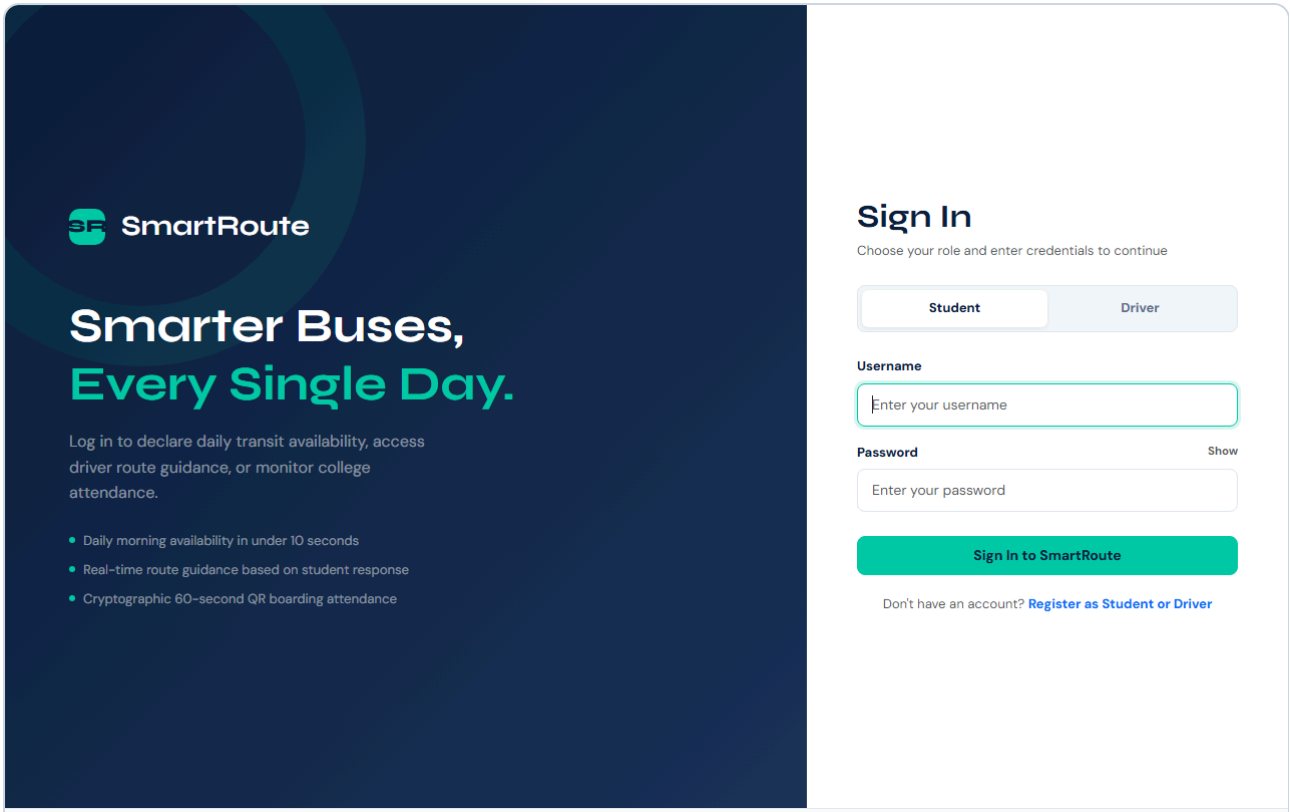


Figure 7.2: Unified Authentication Portal with Role-Based Dispatching

7.3 Student Dashboard & Daily Availability Module

Provides students with real-time controls to toggle daily commute availability, select their active pickup stop, view live bus movements on an interactive Leaflet map, scan the driver's dynamic QR code, and inspect historical attendance logs.

Key Student Features:

- **Availability Switch:** Instant toggle updating `student_availability` for the current date.
- **Active Rider Indication:** Status confirmation reassuring the student their pickup stop is marked on the driver's manifest.
- **QR Attendance Scanner:** Instant mobile camera feed verifying attendance upon boarding the bus.
- **Attendance Log:** Filterable table displaying date, timestamp, bus stop name, and verification status.

7.4 Driver Navigation & Dynamic QR Console

Designed for tablet and smartphone mounts in the bus cab. It organizes stops sequentially, computes live student counts per stop, flags empty stops as bypass candidates, and displays rotating 60-second QR codes.

Driver Operational Features:

- **Stop Optimization Banner:** Instantly displays stops with zero waiting students so drivers avoid congested detours.
- **Rotating QR Generator:** Generates a fresh cryptographic QR code every 60 seconds with an animated countdown dial.
- **Real-Time Headcount:** Incrementing tally showing total boarded students versus registered students for the morning route.

8. Testing, Verification Matrix & Results

8.1 Testing Methodology

The system underwent rigorous testing across four domains: Unit verification of cryptographic functions, Integration testing of REST API endpoints, Database constraint and transaction testing, and Cross-browser responsive UI testing.

8.2 Test Execution Matrix

Test ID	Test Scenario	Input & Execution Steps	Expected Result	Status
TC-01	User Authentication	Submit valid username and password through `/api/auth/login`.	200 OK, session cookie issued, redirects to role dashboard.	PASS
TC-02	Invalid Credentials	Submit incorrect password on login form.	401 Unauthorized, descriptive error banner displayed.	PASS
TC-03	Role Authorization Guard	Directly access `admin-dashboard.html` with student session.	Redirects to `/login.html`; unauthorized access blocked.	PASS
TC-04	Daily Availability Toggle	Toggle bus availability to 'YES' and select 'Chetak Circle'.	Database record updated; waiting count incremented.	PASS
TC-05	Driver Route Optimization	Query `/api/driver/route-summary` with 0 students on Stop 4.	Stop 4 marked as `skip_candidate: true` in manifest.	PASS
TC-06	Dynamic QR Generation	Driver triggers `POST /api/driver/qr/generate`.	QR image rendered, SHA-256 hash stored with 60s TTL.	PASS
TC-07	Valid QR Attendance Scan	Student scans QR within the 60-second window.	200 OK; attendance record committed to database.	PASS
TC-08	Expired QR Attendance Scan	Student attempts scan 65 seconds after generation.	400 Bad Request: "QR token has expired. Scan current code."	PASS
TC-09	Duplicate Attendance Guard	Student scans valid QR code twice on the same calendar day.	409 Conflict: Attendance already recorded today.	PASS

Test ID	Test Scenario	Input & Execution Steps	Expected Result	Status
TC-10	Map Tile Proxy & Cache	Fetch OpenStreetMap tile via `/api/tiles/14/11545/7098.png`.	200 OK image/png; cached in temporary directory.	PASS
TC-11	Serverless DB Reconnect	Trigger API endpoint during database cold-start.	Pool reconnects gracefully without crashing Node process.	PASS
TC-12	Mobile Viewport Fluidity	Test landing page and dashboard on 375px mobile screen.	Components collapse to single column; zero horizontal scroll.	PASS

9. Conclusion & Future Enhancements

9.1 Project Outcomes & Conclusions

SmartRoute successfully demonstrates the practical implementation of end-to-end full-stack web engineering principles as prescribed in the Web Development curriculum. Key project achievements include:

- **Elimination of Static Paper Roll-Calls:** Replaced vulnerable physical sign-in sheets with a cryptographically enforced, 60-second dynamic QR code attendance system.
- **Actionable Fleet Route Optimization:** Drivers receive live stop occupancy data, preventing redundant detours to deserted stops and decreasing trip transit times and fuel waste.
- **Production Cloud Deployment:** Configured for high-speed serverless deployment on Vercel Edge networks with static asset caching, clean routing rewrites, and support for remote cloud MySQL clustering.
- **Open Standards Architecture:** Built using standards-compliant HTML5, CSS3, ES6 JavaScript, Express.js, and normalized MySQL without proprietary framework bloat.

9.2 Future Enhancements

To further extend the capabilities of SmartRoute in subsequent iterations, the following roadmap is proposed:

1. **Automated IoT GPS On-Board Trackers:** Integrate Raspberry Pi / ESP32 OBD-II telemetry hardware directly into bus cabins to transmit automated vehicle coordinates without requiring a driver smartphone screen to remain awake.
2. **Parent & Student SMS Notifications:** Integrate Twilio or Firebase Cloud Messaging (FCM) to push automated arrival notices when the bus is within a 2-kilometer geofence of the student's designated stop.
3. **Machine Learning Route Optimization:** Analyze historical monthly attendance patterns with predictive clustering models (e.g. DBSCAN or K-Means) to dynamically re-sequence bus routes based on day-of-the-week attendance trends.
4. **Offline Attendance Caching (PWA):** Implement Service Workers and IndexedDB client storage to allow offline attendance marking in low-cellular college transit corridors, syncing automatically upon network re-establishment.

10. References & Technical Bibliography

1. **MDN Web Docs:** *HTML5 Semantic Elements and Accessible Form Controls*. Mozilla Developer Network, 2026. https://developer.mozilla.org
2. **W3C Recommendations:** *CSS Flexible Box Layout Module Level 1 & CSS Grid Layout Module Level 2*. World Wide Web Consortium (W3C), 2024. https://www.w3.org/TR/css-flexbox-1/
3. **Express.js Official Documentation:** *Fast, unopinionated, minimalist web framework for Node.js*. Expressjs Foundation, 2026. https://expressjs.com
4. **MySQL 8.0 Reference Manual:** *InnoDB Architecture, ACID Compliance, Connection Pooling, and Constraints*. Oracle Corporation, 2026. https://dev.mysql.com/doc/
5. **Leaflet.js Documentation:** *An open-source JavaScript library for mobile-friendly interactive maps*. Vladimir Agafonkin, 2026. https://leafletjs.com
6. **RFC 7519 / RFC 6234:** *US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)*. Internet Engineering Task Force (IETF).
7. **Vercel Serverless Functions Specification:** *Deploying Express.js Applications on Vercel Node.js Runtime*. Vercel Documentation, 2026. https://vercel.com/docs

---

End of Minor Project Report — SmartRoute College Bus Management & Attendance System